

SENTINEL-NIDS

Network intrusion detection for critical infrastructure — evaluated honestly

Machine-learning intrusion detectors report 97–99% accuracy.

I changed only the train/test split. Accuracy fell to **75.28%**.

Manoj Kharkar · solo submission

ML Bubble 2026 · Army Institute of Technology, Pune · Track: TE-BE (Design & Solve — Advanced)

Round 1 — Idea Submission · 08 August 2026

Critical networks are defended by systems that cannot see a new attack

- Defence networks, power grids and military logistics are still largely protected by signature-based intrusion detection — traffic is flagged only if it matches a known pattern.
- That is structurally blind to any attack never seen before. Machine learning is the obvious replacement.
- Intrusion detection is not pattern matching. It is a distribution-shift problem: the adversary actively invents traffic the model has never seen.
- The rarest attack classes are the most dangerous ones, and a missed intrusion costs incomparably more than a wasted analyst minute.
- So the question that decides whether an ML-IDS number is meaningful is not which model was used. It is how the data was split.

Why the published numbers can't be trusted

Standard supervised learning assumes the test data is drawn from the training distribution.

In intrusion detection that assumption is false by definition — the attacker's job is to violate it.

So the way a benchmark is split is not a technical detail. It decides whether the reported number means anything at all.

THE HYPOTHESIS

The high accuracies reported for ML-based intrusion detection are an artefact of evaluating on a random split of a single dataset.

Under a realistic protocol — where the test set contains attack types absent from training — performance collapses, and the collapse is concentrated precisely in the rarest and most severe attack categories.

This is a falsifiable claim, and the experiment that tests it is one command.

NSL-KDD — chosen because it ships a built-in zero-day benchmark

125,973

training flows (KDDTrain+)

22,544

test flows (KDDTest+)

17

attack types that appear ONLY in the test set

16.6%

of test flows are an attack type never trained on

Two properties — both destroyed by pooling the splits

- 17 of the 37 attack types occur only in KDDTest+ — 3,750 flows. The model must generalise to attacks it has never observed.
- The class priors move: R2L is 0.79% of training data but 12.2% of test data (15.5×); U2R moves 0.04% → 0.89% (21.5×).
- 0 duplicate rows in training — so this is NOT classical row leakage. It is distribution shift, which is a more interesting failure.

Why not a newer dataset?

UNSW-NB15, CIC-IDS2017 and HuggingFace NIDS corpora were all attempted and are unreachable from the build environment (verified 403).

But NSL-KDD is also the right choice on merit: it is the only common benchmark with a documented unseen-attack test protocol — which is what makes the experiment possible.

Its age (1999 traffic) is a genuine limitation, stated in the model card rather than hidden.

One model. One pipeline. One seed. Only the split changes.



PROTOCOL A

The common error

Concatenate KDDTrain+ and KDDTest+ (148,517 rows), shuffle, stratified 80/20 split.

This is what a large share of tutorials, blog posts and student projects do.

Used ONLY for this comparison. Never to report our performance.



PROTOCOL B

Honest

Train on KDDTrain+ (125,973). Test on KDDTest+ (22,544) — as the dataset authors designed it.

The test set keeps its 17 unseen attack types and its shifted class priors.

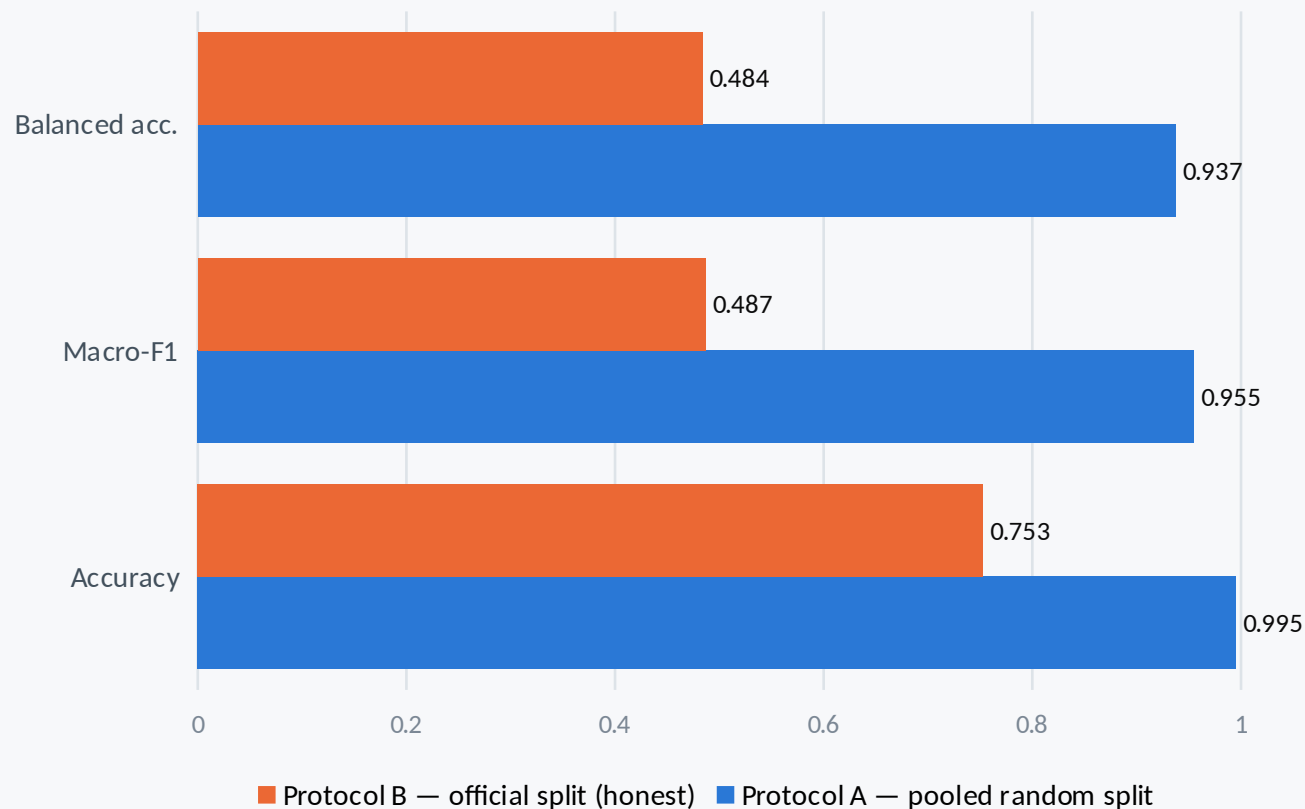
Every performance number we claim comes from here.

Held constant across both protocols

RandomForestClassifier(n_estimators=300) · 122 one-hot features · identical preprocessing · random_state=42 · no tuning on either split

THE RESULT

Accuracy falls 24 points. Macro-F1 falls 47. The error rate is 50× higher.



0.995

accuracy reported under
Protocol A

0.753

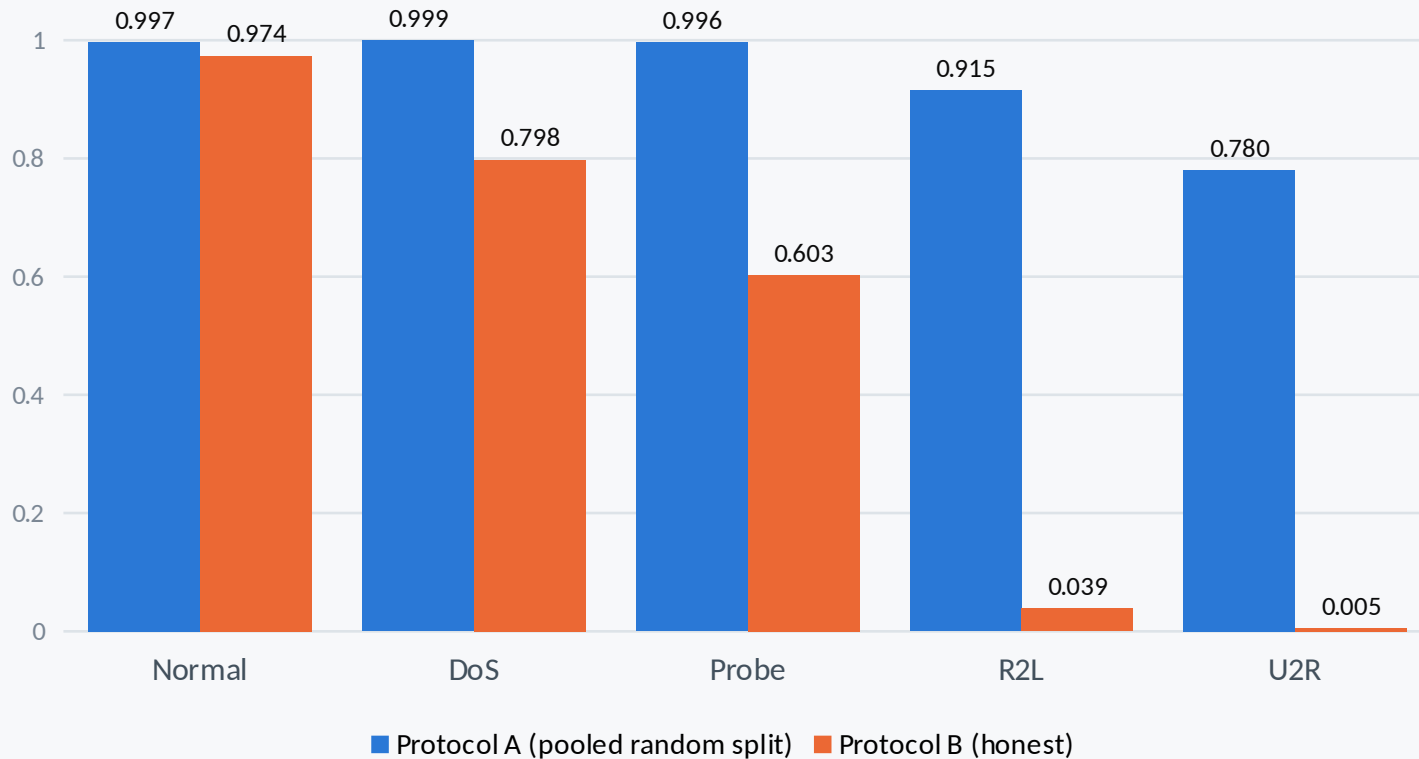
accuracy under the honest
protocol

50.4× more errors

under honest evaluation, same model

Nothing about the model changed. Only the way it was measured.

The collapse lands on the two families that mean an attacker got in



Recall, by attack family

R2L — remote-to-local intrusion

0.915 → 0.038

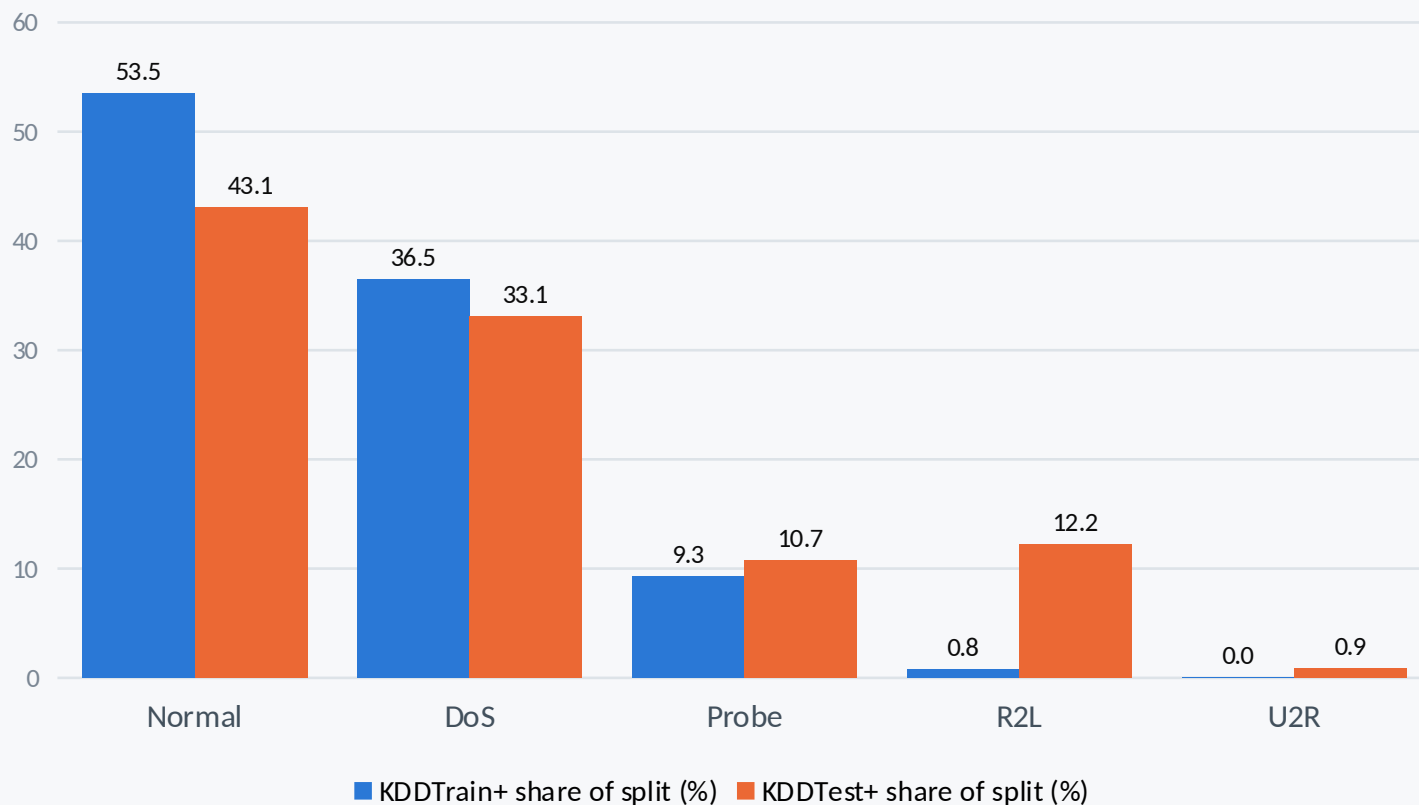
2,754 of these in the test set. The model finds 106.

U2R — privilege escalation

0.780 → 0.005

200 in the test set. The model finds one.

Why: the test set is not the training distribution — by design



15.5×

more R2L in test than in training (0.79% → 12.2%)

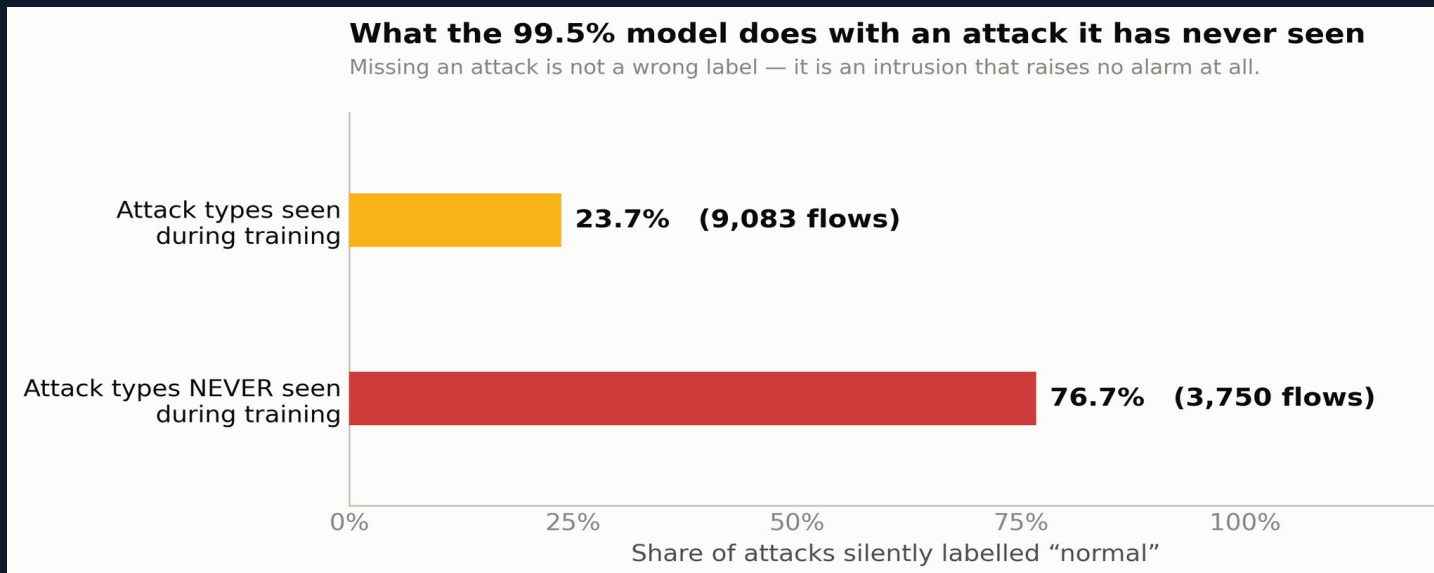
21.5×

more U2R in test than in training (0.04% → 0.89%)

Pooling the two files forces the priors to match — and deletes the whole problem.

THE OPERATIONAL CONSEQUENCE

A missed attack is not a mislabelled row. It is an intrusion that raises no alarm at all.



76.7%

of attack types never seen in training are silently cleared as "normal"

18.6%

of unseen attacks are assigned the correct attack family

The model that reports 99.5% accuracy is, against a genuinely new attack, close to useless — and it says nothing to warn you.

Eight models, identical splits — the protocol dominates the model

Model	Macro-F1 (honest)	CV F1 (in-dist.)	Shift penalty	R2L recall	Unseen missed
mlp	0.546	0.889	0.343	0.106	51%
logistic regression	0.540	0.840	0.300	0.037	58%
xgboost	0.528	0.940	0.412	0.060	62%
random forest	0.494	0.910	0.415	0.048	76%
decision tree	0.487	0.912	0.425	0.069	59%
random forest balanced	0.468	0.904	0.435	0.005	86%
lightgbm	0.460	0.322	-0.138	0.046	76%
baseline majority	0.120	0.139	0.019	0.000	100%

Three findings

1

The protocol beats the model.

Every real model scores 0.84–0.94 in-distribution and 0.46–0.55 under shift. The gap between columns (~0.40) dwarfs the gap between best and worst model (~0.09).

2

Capacity does not help.

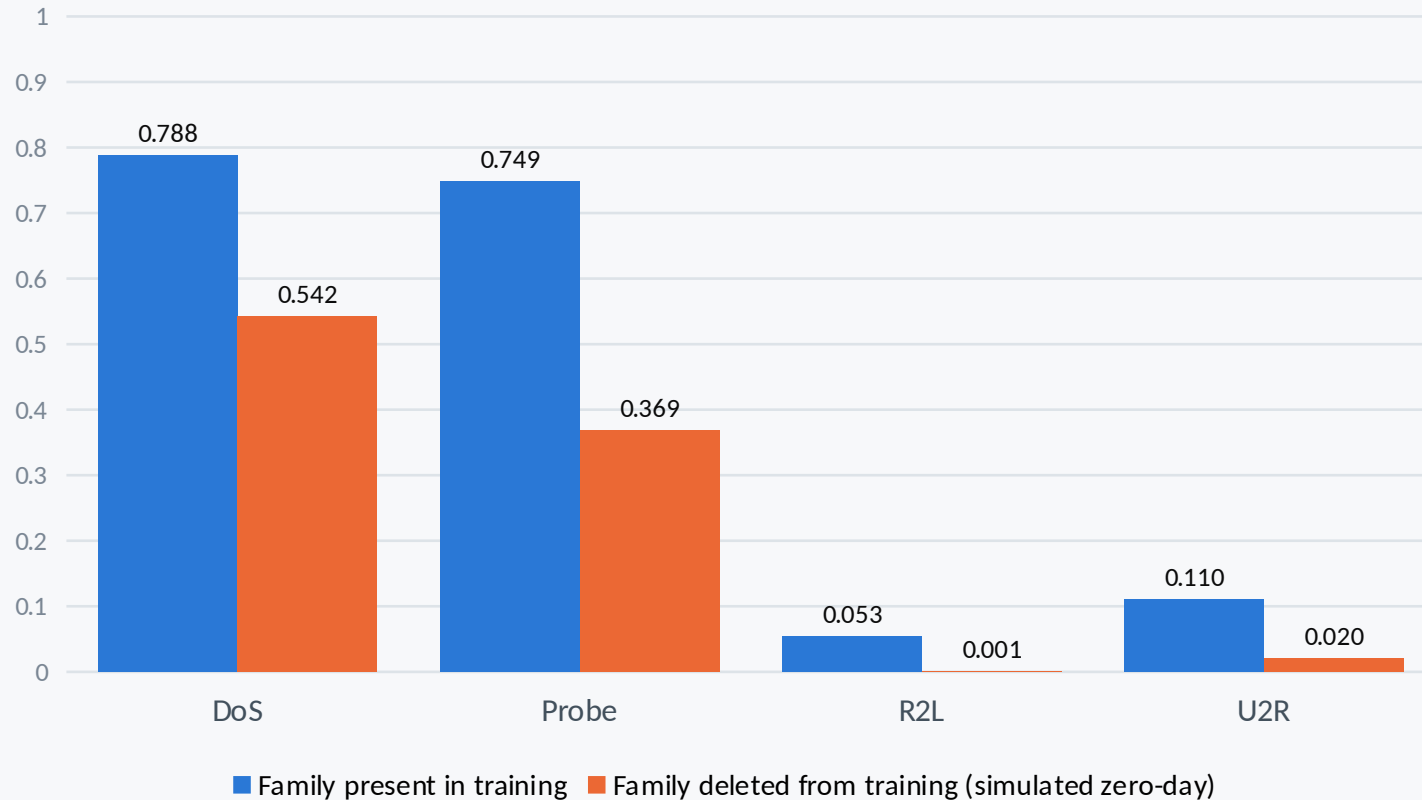
The two simplest models — a small MLP and plain logistic regression — generalise best. XGBoost has the highest CV score and a worse test score than logistic regression.

3

Rebalancing makes it worse.

`class_weight='balanced_subsample'` cut macro-F1 from 0.494 to 0.468 and R2L recall from 0.048 to 0.005. Reweighting cannot invent information about absent attack types.

Controlled zero-day: delete an attack family, then test on it



Detection rate on the held-out family. Each bar pair is a model retrained from scratch.

Why run this as well?

The test-set result confounds two things: novelty and the prior shift. This isolates novelty.

For each family: delete every one of its rows from training, refit from scratch, then measure detection on it.

Probe detection falls 0.749 → 0.369 purely from removing Probe. The novelty effect is real and not an artefact of the priors.

0.369

Probe detection when Probe is unseen (0.749 when seen)

The fix: give the detector a way to say “I don't recognise this”

A softmax over five known classes cannot express “none of these”. Forced to choose, it picks the class with the largest prior mass — Normal. The failure is structural, not a tuning problem.



Channel A — Supervised + abstention

Logistic regression over the 5 known families.

If the argmax is Normal but confidence $< \tau$, do not clear the flow — emit SUSPICIOUS_UNCLASSIFIED and route to an analyst.



Channel B — Unsupervised novelty

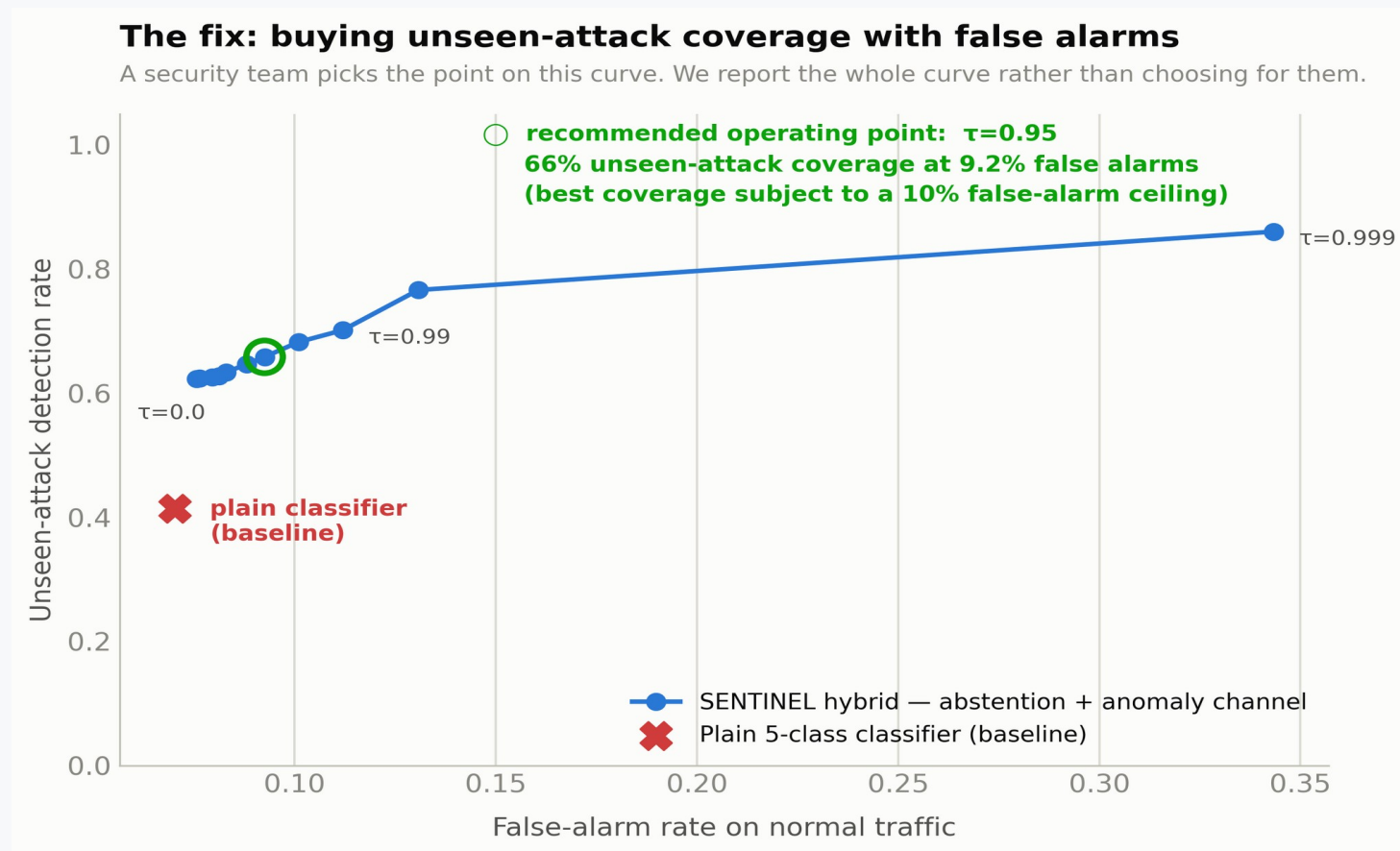
IsolationForest fitted on the 67,343 NORMAL training flows only.

It never sees a single attack during training, so it cannot be biased toward the 22 known attack types.

```
alarm = A says attack OR A abstains OR B says outlier
verdict ∈ { NORMAL , ATTACK:family , SUSPICIOUS_UNCLASSIFIED }
```

Configuration	Unseen-attack detection	Seen-attack detection	False-alarm rate
Channel A alone (plain 5-class classifier)	0.415	0.756	0.070
Channel B alone (trained on zero attacks)	0.566	0.638	0.020
A OR B, no abstention	0.624	0.770	0.076
A + abstention ($\tau=0.95$) OR B — recommended	0.659	0.804	0.092

Unseen-attack coverage rises 24 points for 2 points of false alarms



41.5%

unseen attacks caught — plain classifier

65.9%

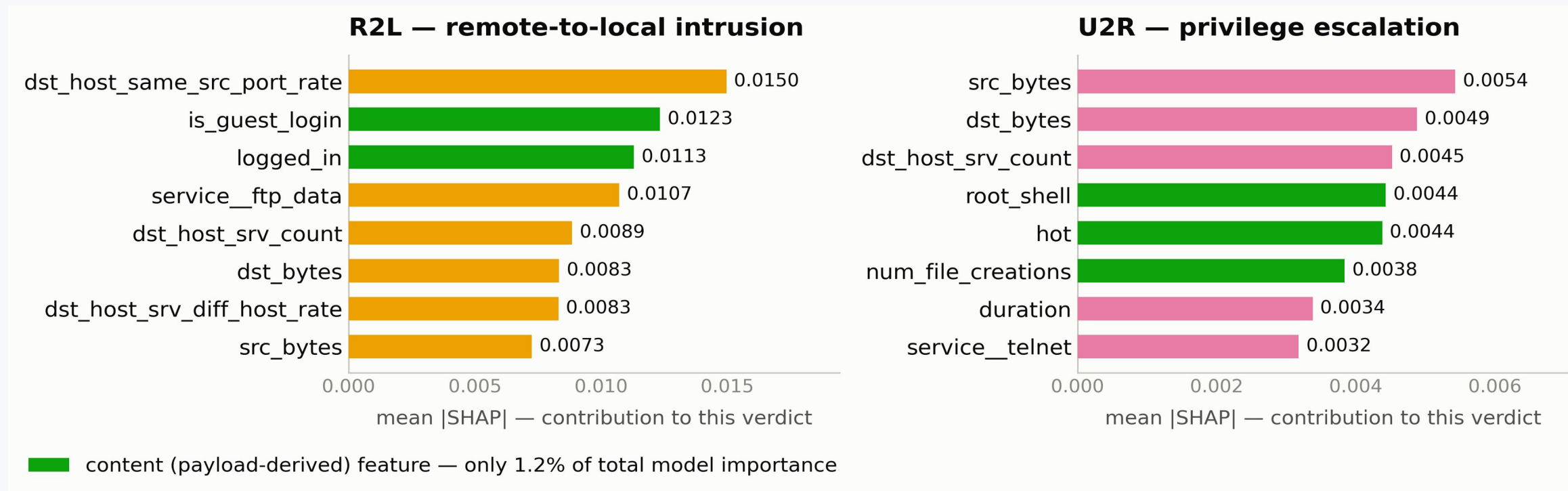
unseen attacks caught — SENTINEL hybrid

+24.4 pp

coverage gained, for only +2.23 pp of false alarms

An unsupervised model that never saw a single attack detects unseen attacks better than the supervised one — 0.566 vs 0.415.

Why R2L and U2R fail — a mechanistic answer, not a shrug



1.23%

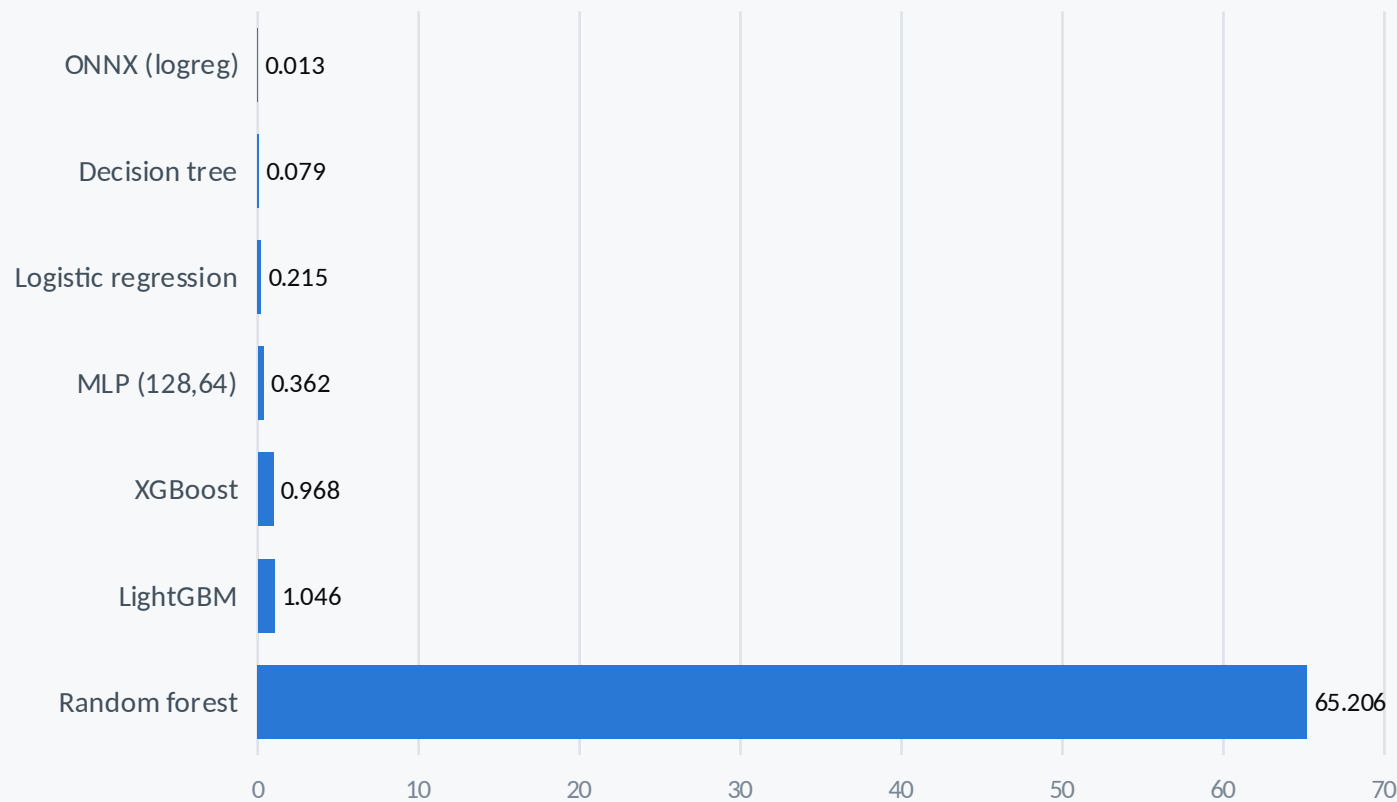
of total model importance sits in CONTENT features

51.72%

sits in basic header / byte-count features

The model became a traffic-pattern detector, because that is 99% of its training signal. A successful password guess just looks like one ordinary login.

Deployment: the best model is also the cheapest to run



p50 inference latency per flow (ms). Random forest is off-scale at 65.206 ms.

0.0129 ms

per flow, ONNX Runtime single-threaded

4.9 KB

deployed model size on disk

810k flows/s

single-threaded throughput — about 8× a saturated 1 Gbps link

Random forest — the reflexive choice — is 5,055× slower and 8,027× larger, for 0.05 LOWER macro-F1.

WHAT THIS PROJECT DOES NOT CLAIM

Stated here rather than buried

Dataset age

NSL-KDD is 1999-era simulated traffic. The methodological finding transfers to any dataset with a designed shift; the absolute numbers do not describe a 2026 network.

Single dataset

The strongest remaining criticism. Cross-dataset validation was planned; UNSW-NB15 and CIC-IDS2017 hosts are unreachable from the build environment.

The fix is partial

Even at the recommended operating point, 34% of unseen attacks are still missed. That is a real gap, not a rounding error.

R2L and U2R remain unsolved

Best-case recall 0.106 and 0.105 across all eight models. We explain the cause mechanically; we do not fix it.

No adversarial robustness

We evaluate against a fixed dataset, not an adaptive adversary. A linear model is easy to evade with white-box knowledge.

Feature extraction out of scope

Latency figures measure inference on pre-computed flow aggregates. Computing them from live packets is the harder engineering problem.

For a defence evaluator, a system whose limits are known is worth more than one whose headline number cannot be trusted.

Everything here is reproducible

Every number on every slide is read from a JSON file in reports/metrics/. None was typed by hand.

Reproduce the whole project

```
git clone <repo> && cd sentinel-nids  
pip install -r requirements.txt  
python run_all.py
```

Reproduce just the headline

```
python run_all.py --only leakage
```

≈ 35 seconds. Prints both protocols
and the gap between them.

Submitted artefacts

Source code (9 modules, all idempotent and checkpointed) · dataset provenance + SHA-256 manifest · 6 metrics JSON files · 12 figures · README, methodology, model card, deployment notes · FastAPI inference service · 4.9 KB ONNX model · this deck

A detector that reports 99.5% accuracy silently clears 77% of never-before-seen attacks. Measuring it honestly is the whole job.